Step-by-Step Guide: ASG + ALB with Path-Based Routing

🔗 1. Create EC2 Instances with Apache & Unique Webpages

Do this **twice** (for Server1 and Server2):

For Each EC2:

- **AMI**: Amazon Linux 2
- **Security Group**: Allow **HTTP (80)** and **SSH (22)**
- **User Data (Apache + HTML)**:

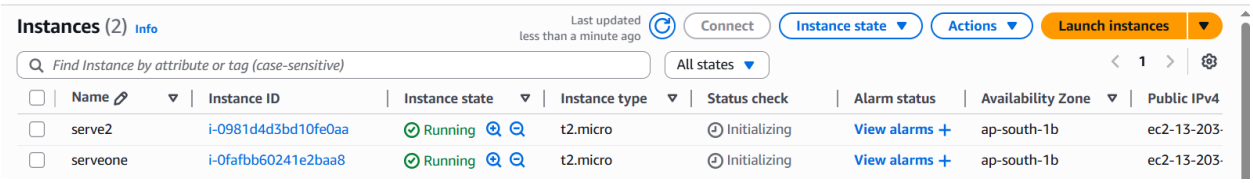
bash

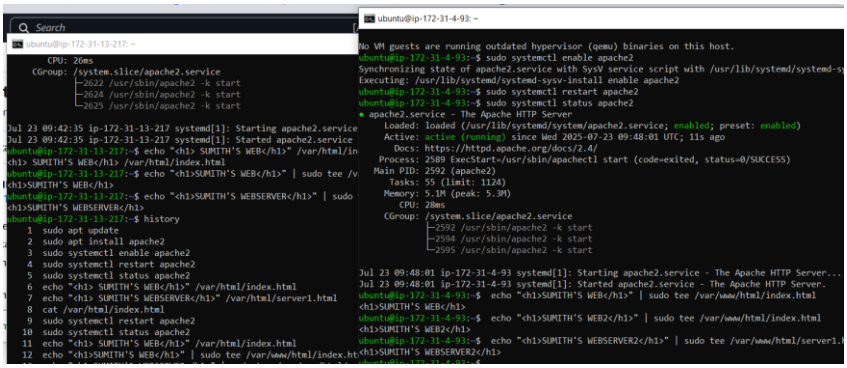
CopyEdit

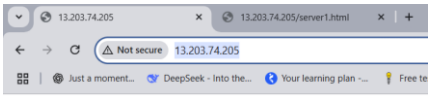
```
#!/bin/bash
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd
echo "<h1>This is Server1</h1>" > /var/www/html/index.html
echo "<h1>This is Server1 - server1.html</h1>" > /var/www/html/server1.html
```

(Change text for Server2 accordingly)

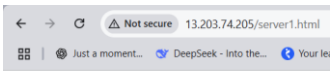
- **Test in Browser**: http://<EC2_Public_IP>/server1.html



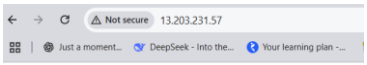




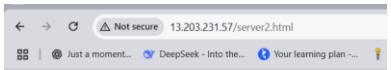
SUMITH'S WEB



SUMITH'S WEBSERVER



SUMITH'S WEB2



SUMITH'S WEBSERVER2

2. Create AMIs from Both Instances

- **Go to EC2 > Instances > Actions > Image > Create Image**
- Name them: AMI-Server1, AMI-Server2

Image details

Instance ID

i-0fafbb60241e2baa8 (serveone)

Image name

imgone

Maximum 127 characters. Can't be modified after creation.

Image description - *optional*

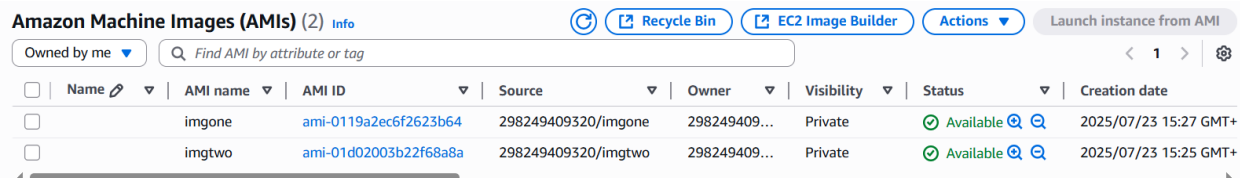
Instance ID

i-0981d4d3bd10fe0aa (serve2)

Image name

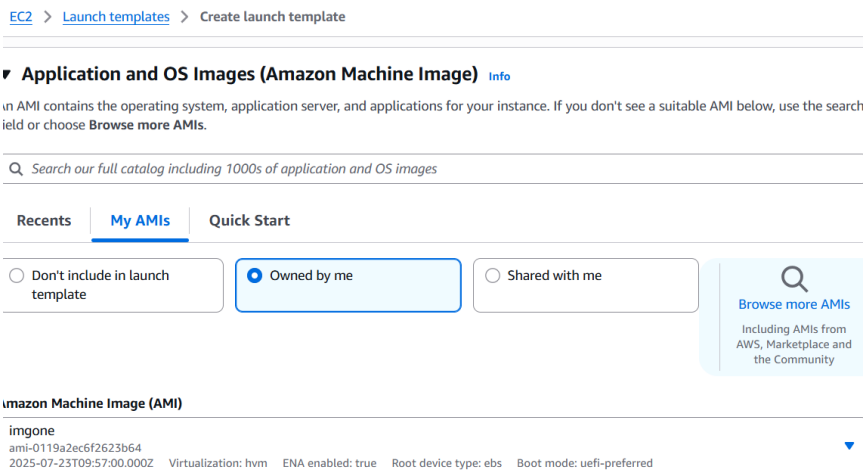
imgtwo

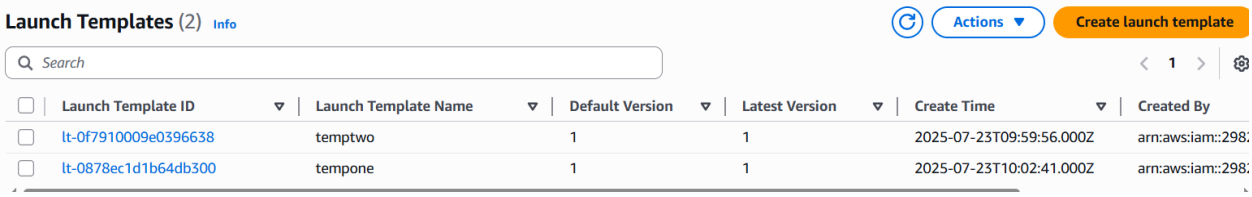
Maximum 127 characters. Can't be modified after creation.



3. Create Launch Templates

- Go to **EC2 > Launch Templates > Create launch template**
- Name: LT-Server1, LT-Server2
- Use the corresponding **AMI**
- Set instance type, key pair, security group, etc.

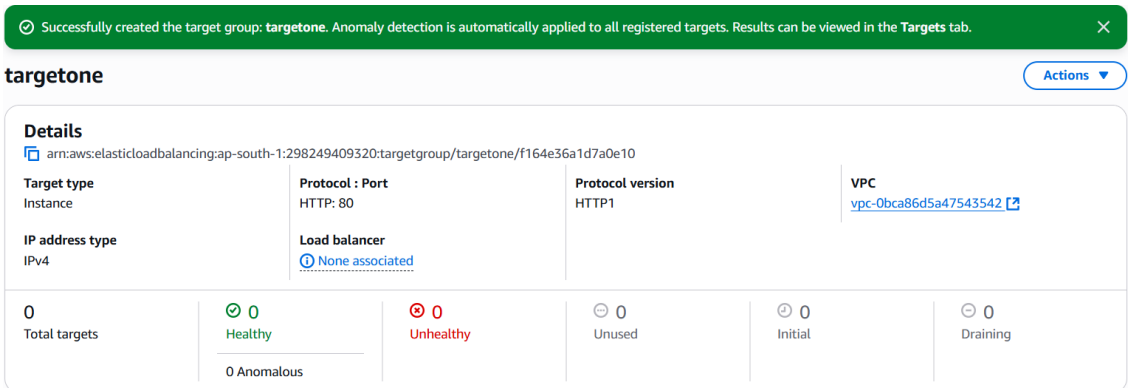


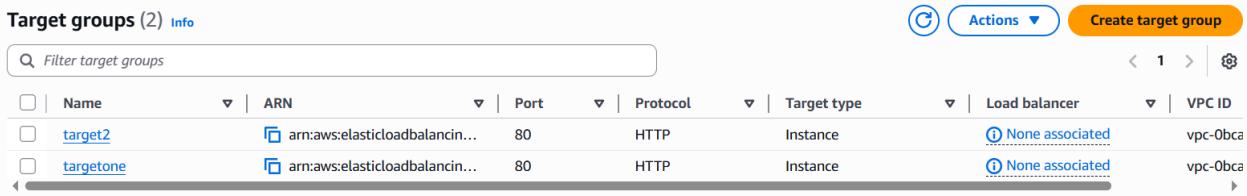


4. Create Two Target Groups

- Go to **EC2 > Target Groups > Create target group**
- Type: **Instances**
- Protocol: HTTP (port 80)
- Target type: **instance**
- Name: TG-Server1, TG-Server2

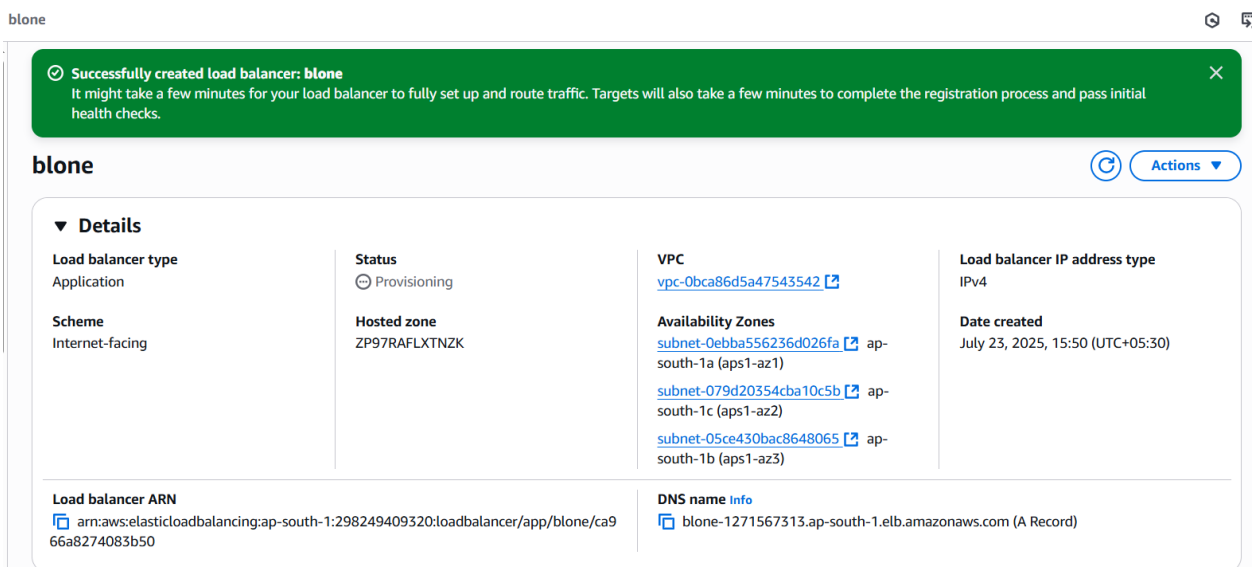
(Leave targets empty – ASG will register automatically)





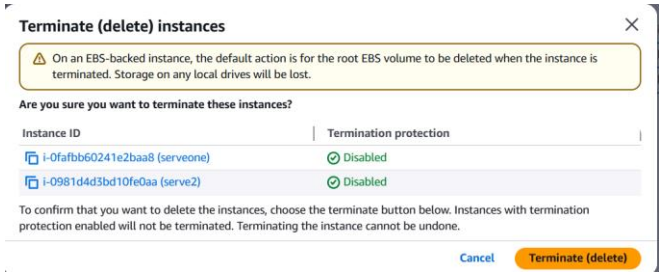
5. Create Application Load Balancer

- Go to **EC2 > Load Balancers > Create Load Balancer > Application Load Balancer**
- Name: MyALB
- Scheme: Internet-facing
- Listeners: HTTP (Port 80)
- Add at least **2 subnets in different AZs**
- Security group: Allow port 80



6. Create Two Auto Scaling Groups

- Go to **EC2 > Auto Scaling Groups > Create Auto Scaling Group**
- Name: ASG-Server1, ASG-Server2
- Use corresponding launch template
- Network: Select your VPC + 2 subnets
- Attach to the corresponding **Target Group**
- Set Min: 1 | Max: 2 | Desired: 1



7. Add Path-Based Routing Rules

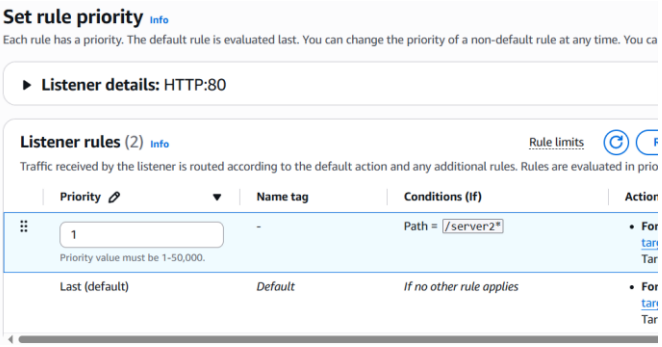
- Go to **ALB > Listeners > View/Edit Rules**
- Default Rule: Optional (send to one target group or fixed response)
- Add 2 rules:

  pgsql

  CopyEdit

  IF path is /server1.html THEN forward to TG-Server1

  IF path is /server2.html THEN forward to TG-Server2



◈ 1. Advanced Auto-Scaling Policies

✅ Dynamic Scaling Policies:

Goal: Automatically scale EC2 instances based on CPU utilization or ALB request count.

Steps:

1. **Create a Launch Template** with a user-data script to install a web server.
2. **Create Auto Scaling Group (ASG)**:
    - Attach to **Application Load Balancer (ALB)**.
    - Enable scaling policies.

Example Scaling Policy:

- **CPU-based Scaling**:

  TargetTrackingPolicy → Target Value: 70%
- **Request-based Scaling**:

  Scale on ALB → requests > 100 per target per minute

Cool-down Periods:

- Modify ASG to add:
    - **Default Cooldown**: 300 seconds
    - **Scale-In Cooldown**: 300 seconds
    - Prevent rapid scale-in/out reactions.

◈ 2. Cost Optimization

✅ Compare Linux vs Windows EC2:

- **Linux t2.micro** (Free Tier eligible): ~$0.0116/hr
- **Windows t2.micro**: ~$0.0200/hr

✅ AWS Pricing Calculator:

- Go to https://calculator.aws.amazon.com
- Estimate:
    - ALB: $0.0225/hr + $0.008 per LCU
    - EC2: t2.micro (~$8.50/month)
    - 1000 requests/hour = 24,000/day ≈ 0.03 LCU

📌 **Estimated monthly** (Linux setup): ~$20–25/month for ALB + ASG

◈ 3. High Availability (HA) Testing

✅ **Simulate EC2 Failures:**

- Manually terminate an EC2 instance in ASG.
- Check: ASG replaces it automatically (heals within ~2 mins).

✅ **Cross-AZ Deployment:**

- Select **2+ Availability Zones** in ASG configuration.
- Outcome:
  - Instances launched across zones
  - Fault tolerance if one AZ fails

◈ 4. Security Enhancements

✅ **Custom Security Group:**

- Allow HTTP (80) only from specific IP range (e.g., your office).
- Deny SSH from the world — allow only from jump host or specific IP.

bash

CopyEdit

```
Inbound:
  - TCP 80 from x.x.x.x/32
  - TCP 443 from x.x.x.x/32
Outbound:
  - Allow all
```

✅ **Enable HTTPS using ACM:**

1. Go to **AWS Certificate Manager (ACM)**.
2. Request a public certificate (e.g., *.example.com).
3. Attach SSL cert to **ALB Listener**.
4. Forward HTTPS to backend on port 80 or 443.

◈ 5. Automation with Infrastructure as Code (IaC)

🛠 **Terraform Example (basic ASG + ALB)**

hcl

CopyEdit

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_launch_template" "web_lt" {
  name_prefix   = "web-lt"
  image_id      = "ami-0abcdef1234567890"
  instance_type = "t2.micro"

  user_data = base64encode(<<-EOF
            #!/bin/bash
            sudo apt update
            sudo apt install -y apache2
            sudo systemctl start apache2
            EOF
  )
}

resource "aws_autoscaling_group" "asg" {
  desired_capacity     = 2
  max_size             = 4
  min_size             = 1
  vpc_zone_identifier  = ["subnet-abc", "subnet-def"]
  launch_template {
    id      = aws_launch_template.web_lt.id
    version = "$Latest"
  }
  target_group_arns    = [aws_lb_target_group.tg.arn]
}

resource "aws_lb" "alb" {
  name               = "web-alb"
```

```
  internal           = false
  load_balancer_type = "application"
  subnets            = ["subnet-abc", "subnet-def"]
  security_groups    = [aws_security_group.alb_sg.id]
}

resource "aws_lb_target_group" "tg" {
  name     = "web-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = "vpc-abc123"
}

resource "aws_lb_listener" "listener" {
  load_balancer_arn = aws_lb.alb.arn
  port              = 80
  protocol          = "HTTP"
  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.tg.arn
  }
}
```

🛠 **AWS CDK (Python):**

python

CopyEdit

```
from aws_cdk import (
    aws_autoscaling as autoscaling,
    aws_ec2 as ec2,
    aws_elasticloadbalancingv2 as elbv2,
    core
)
```

```
class WebStack(core.Stack):
    def __init__(self, scope: core.Construct, id: str, **kwargs):
        super().__init__(scope, id, **kwargs)

        vpc = ec2.Vpc.from_lookup(self, "Vpc", is_default=True)
        lt = autoscaling.LaunchTemplate(self, "LaunchTemplate",
            instance_type=ec2.InstanceType("t2.micro"),
            machine_image=ec2.MachineImage.latest_amazon_linux()
        )

        asg = autoscaling.AutoScalingGroup(self, "ASG",
            vpc=vpc,
            launch_template=lt,
            min_capacity=1,
            max_capacity=4
        )

        lb = elbv2.ApplicationLoadBalancer(self, "ALB",
            vpc=vpc,
            internet_facing=True
        )

        listener = lb.add_listener("Listener", port=80)
        listener.add_targets("AppFleet", port=80, targets=[asg])
```

📌 **Best Practices**

- Tag all resources properly.
- Set up CloudWatch alarms for CPU, HTTP 5xx errors.
- Enable access logs on ALB.
- Use Route 53 health checks for DNS-level failover.
- Backup Terraform state (if using Terraform) in S3 with locking via DynamoDB.